

Clase 227 — Seguridad de contenedores: Docker

Parte: **10 — Seguridad en la nube y contenedores** · Fuente: *Liz Rice, "Container Security" (O'Reilly) y CIS Docker Benchmark* 🕒 Duración estimada: **130 min** · Nivel: **Intermedio**

Objetivo

Entender cómo se aísla realmente un contenedor (namespaces, cgroups, capabilities) y aplicar seguridad en las tres fases del ciclo de vida: construcción de imágenes, almacenamiento/registro y ejecución. Al terminar, el alumno podrá escanear imágenes con Trivy, endurecer un Dockerfile y configurar el runtime según el CIS Docker Benchmark.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** el aislamiento de contenedores (namespaces, cgroups, capabilities) y sus límites.
2. **Construir** imágenes mínimas y sin secretos con multi-stage builds.
3. **Escanear** imágenes en busca de CVEs y secretos con Trivy.
4. **Ejecutar** contenedores con privilegio mínimo (usuario no root, capabilities recortadas, read-only).
5. **Auditar** un host Docker con Docker Bench for Security.

Temas

#	Tema	Por qué importa
1	Namespaces y cgroups	Base del aislamiento; no es una VM
2	Capabilities y user namespaces	Reducir privilegios del proceso
3	Imágenes y capas	Superficie de vulnerabilidades y secretos
4	Dockerfile seguro y multi-stage	Imágenes pequeñas y limpias
5	Escanear de imágenes (Trivy)	Detectar CVEs antes de desplegar
6	Runtime seguro (seccomp, AppArmor)	Contener el proceso en ejecución
7	Registro y firma de imágenes	Cadena de suministro confiable

Definiciones y características

- **Namespace:** aísla la vista de un proceso (PID, red, montajes, usuarios). *Clave:* es el mecanismo central de aislamiento del contenedor.
- **cgroup:** limita recursos (CPU, memoria, PIDs). *Clave:* previene abuso de recursos y algunas denegaciones de servicio.
- **Capability:** privilegio granular del kernel (p. ej. `CAP_NET_ADMIN`). *Clave:* recórtalas con `--cap-drop=ALL` y añade solo las necesarias.

- **Contenedor privilegiado:** `--privileged` desactiva casi todo el aislamiento. *Clave:* casi equivale a root en el host; evítalo.
- **Multi-stage build:** compilar en una etapa y copiar solo el artefacto a una imagen mínima. *Clave:* elimina toolchain y secretos de la imagen final.
- **Distroless / scratch:** imágenes base sin shell ni gestor de paquetes. *Clave:* reducen superficie de ataque.
- **Seccomp:** filtra syscalls disponibles al contenedor. *Clave:* el perfil por defecto ya bloquea syscalls peligrosas.

Herramientas y preparación

- Docker Engine en un host Linux de laboratorio.
- **Trivy** para escaneo de imágenes/filesystem: `docker run aquasec/trivy`.
- **Docker Bench for Security** para auditar el host según CIS.
- **Hadolint** para lint de Dockerfiles.

```
# Escanear una imagen en busca de CVEs y secretos
trivy image --severity HIGH,CRITICAL nginx:latest
# Ver capabilities y perfil seccomp de un contenedor en ejecución
docker inspect --format '{{ .HostConfig.CapAdd }} {{ .HostConfig.SecurityOpt }}' mi_contenedo
```

Laboratorio guiado

1. Ejecuta `docker run -it --rm alpine sh` y explora los namespaces con `lsns` desde el host para ver el aislamiento.
2. Lanza un contenedor con `--privileged` y muestra que puede montar el disco del host; borra el contenedor y **nunca uses privileged en producción**.
3. Escribe un Dockerfile inseguro (corriendo como root, con secretos en `ENV`) y pásalo por **Hadolint**; corrige los hallazgos.
4. Refactoriza a un **multi-stage build** con imagen final `distroless` o `scratch`, usuario no root (`USER 1000`) y sin secretos.
5. Escanea ambas imágenes con `trivy image` y compara el número de CVEs y el tamaño.
6. Ejecuta el contenedor endurecido: `docker run --read-only --cap-drop=ALL --security-opt=no-new-privileges --user 1000 mi_imagen`.
7. Audita el host con **Docker Bench for Security** y corrige al menos tres hallazgos (daemon con TLS, `userns-remap`, logging).

Ejercicios

1. Enumera las capabilities por defecto de un contenedor y explica cuáles quitarías.
2. Convierte una imagen basada en `ubuntu` a una `distroless` y mide la reducción de CVEs.
3. Configura un contenedor con sistema de archivos read-only y un volumen escribible acotado.
4. Escribe un `.dockerignore` que evite filtrar `.env` y `.git`.
5. Firma una imagen y verifica su firma antes de desplegar.
6. Explica por qué un contenedor comprometido puede afectar al host y cómo mitigarlo.

Reto verificable

Toma una imagen de aplicación real y prodúcela endurecida: multi-stage, base mínima, usuario no root, sin secretos, y ejecución con capabilities recortadas y read-only.

Criterio de aceptación: `trivy image` no reporta CVEs CRÍTICAS ni secretos en la imagen final, la imagen corre como UID no root, y `docker inspect` confirma `CapDrop: [ALL]` y `ReadOnlyRootfs: true`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>permission denied</code> al escribir en read-only	El proceso necesita escribir; monta un volumen/tmpfs acotado para esa ruta.
Secreto visible en <code>docker history</code>	Se pasó por <code>ENV</code> / <code>ARG</code> o <code>COPY</code> ; usa build secrets o inyección en runtime.
Contenedor corre como root sin querer	Falta <code>USER</code> en el Dockerfile; añádelo y ajusta permisos de archivos.
Trivy reporta cientos de CVEs	Imagen base gorda/antigua; cambia a base mínima y actualiza.
<code>--privileged</code> "necesario" para que funcione	Casi nunca lo es; identifica la capability concreta y añádela sola.

Preguntas frecuentes

 **¿Un contenedor es tan seguro como una máquina virtual?** No. Comparten el kernel del host; una fuga o un contenedor privilegiado pueden comprometer el host. Para aislamiento fuerte se usan sandboxes como gVisor o Kata Containers.

 **¿Por qué correr como no root si el contenedor ya está aislado?** Porque si el atacante escapa del contenedor o explota una capability, ser root dentro facilita el escape hacia el host. El usuario no root reduce el impacto de un compromiso.

 **¿Dónde guardo los secretos si no en el Dockerfile?** Fuera de la imagen: en un gestor de secretos (clase 233), variables inyectadas en runtime o montajes de secretos del orquestador. Nunca en capas de la imagen.

Referencias

- Liz Rice, *Container Security*, O'Reilly. <https://www.oreilly.com/library/view/container-security/9781492056690/>
- CIS Docker Benchmark. <https://www.cisecurity.org/benchmark/docker>
- Trivy. <https://github.com/aquasecurity/trivy>
- Docker Bench for Security. <https://github.com/docker/docker-bench-security>
- NIST SP 800-190, Application Container Security Guide. <https://csrc.nist.gov/pubs/sp/800/190/final>

Siguiente clase

Clase 228 - Seguridad de Kubernetes: arquitectura